

Advanced Algorithms:

Lecture 11 Notes

Dana Randall Spring 2024

Sarthak Mohanty

Ski-Rental Problem

Let's get some more practice with online algorithms. A person is going skiing for an unknown number of days t . Each day, they can either rent skis for R , or they can buy skis for B . If they rent skis, the person is presented with the choice the next day; if they buy, the game ends.

Since the game ends as soon as one buys, any algorithm for this problem can uniquely be defined with one number k where $1 \leq k \leq t$, indicating the day where one buys.

Now suppose $k = B$. Then the total maximum cost is $k - 1 + B = 2B - 1$. In this cost, the offline cost is $\min(t, B) = B$. Thus, the competitive ratio is $(2B - 1)/B = 2 - \frac{1}{B}$.

Proof of optimality: Let k be any strategy (buy after $k - 1$ rents).

- Suppose you buy the skis at the k -th time and then break your leg and never ski again.
- Your total ski cost is $k - 1 + B$ and the optimum offline cost is $\min(k, B)$.
- For every k , the ratio $\frac{k-1+B}{\min(k,B)}$ is at least $(2 - \frac{1}{B})$.
- Therefore, every strategy is at least $(2 - \frac{1}{B})$ -competitive.

The general rule: when balancing small incremental costs against a big one-time cost, you want to delay spending the big cost until you have accumulated roughly the same amount in small costs.

Random Algorithms

Theorem 1.1. *There exists a random algorithm that is $(\frac{e}{e-1})$ -competitive for the ski-rental problem.*

Proof. (Sketch) We assume there is a probability distribution p_t associated with each day t , such that there is always a probability p_t that you buy on day t . We must then calculate the theoretical value of $E[A(I)]$, $OPT(I)$ for every instance I . Then find the worst instance I^* and optimize the probabilities p for

that instance.

We'll consider a simpler version of the problem where $b = 2$ and $l \leq 3$. The table below shows the competitiveness of each algorithm A_i in each instance I_j , where i determines which day to buy, and j determines the last day of the ski tracks. Each entry of the table is the ratio between the algorithm and the optimal solution for that instance, i.e. $\frac{A_i(I_j)}{OPT(I_j)} \leq \alpha$.

	A_1	A_2	A_3	A_4
I_1	2	1	1	1
I_2	1	$\frac{3}{2}$	1	1
I_3	1	$\frac{3}{2}$	2	$\frac{3}{2}$

The best deterministic algorithms are A_2 and A_4 that are both $\frac{3}{2}$ -competitive. If we mix algorithms we could potentially improve the competitiveness. We can mix the two best algorithms. Consider that the algorithm A runs algorithm A_2 and A_4 , each with $\frac{1}{2}$ probability. Algorithm A has the following results for the different instances:

A	
I_1	1
I_2	$\frac{5}{4}$
I_3	$\frac{3}{2}$

It is still $\frac{3}{2}$ -competitive. We were unable to improve on the previous result. Note how algorithms A_2 and A_4 are simultaneously costly (in relation to the optimal) on instance I_3 . Ideally, we'd choose two or more algorithms that are complementary to each other.

For this next try, we'll be choosing algorithms A_1 and A_2 , note how they are complementary to each other. This time, we will not be attributing a priori probabilities to them. Let algorithm A_1 occur with probability p and algorithm A_2 occur with probability $1 - p$. We have the following results for the algorithm A .

A	
I_1	$p + 1$
I_2	$\frac{3-p}{2}$
I_3	$\frac{3-p}{2}$

To optimize the probability p for our mixed strategy against a worst-case adversary, we apply the **Principle of Indifference**. We want to choose p such that our algorithm's expected competitive ratio is identical across the relevant worst-case instances I_1 and I_2 . By setting the expected ratios equal, $A(I_1) = A(I_2)$, we get:

$$p + 1 = \frac{3 - p}{2}$$

Solving this yields $p = \frac{1}{3}$.

By taking the expected value of A , we obtain that the algorithm A is $\frac{4}{3}$ -competitive, a strict improvement on our previous findings.

Main idea: Yao's MINIMAX principle gives "reduction" to the deterministic case:

Randomized algorithms on worst-case deterministic instance $\approx$ Average-case complexity of deterministic algorithms

A randomized algorithm can be thought of as

$$A = \left\{ \begin{array}{ccc} A_1 & A_2 & \dots \\ p_1 & p_2 & \dots \end{array} \right\},$$

where A_i is a deterministic algorithm and A has probability p_i of running algorithm A_i . We want to show that there is no randomized algorithm that is better than c^{-1} -competitive, e.g. there are no randomized algorithm that is 1.1-competitive, equivalently:

For every randomized algorithm A , there exists I such that

$$\mathbb{E}[A(I)] > 1.1 \times \text{OPT}(I)$$

$$\Leftrightarrow \sum p_i A_i(I) > 1.1 \text{OPT}(I)$$

A random instance I can be thought of as

$$I = \left\{ \begin{array}{ccc} I_1 & I_2 & \dots \\ q_1 & q_2 & \dots \end{array} \right\},$$

where I_j is a deterministic instance and I has probability q_i of instantiating to I_j .

Theorem 1.3. (Yao’s MINIMAX principle) For any randomized algorithm A and random instance I as detailed above we have

$$\max_I \frac{\mathbb{E}[A(I)]}{\text{OPT}(I)} \geq \min_A \mathbb{E}_I \left[\frac{A(I)}{\text{OPT}(I)} \right]$$

Where on the left side we’re running the randomized algorithm A over the worst fixed instance and right side we’re running the best deterministic algorithm over a random instance.

Observation 1. Under technical conditions the inequality becomes equality (non-trivial, uses, e.g., von Neumann’s Minimax).

Example: Let’s apply this to our matrix example. To prove a strong lower bound, we must act as the adversary and construct a worst-case probability distribution I over the instances to maximize the *minimum* expected ratio across all algorithms.

Notice that instance I_2 is strictly weaker than I_3 at punishing algorithms that buy late. Therefore, a smart adversary will ignore I_2 and focus on a mixture of the extreme cases: the short track (I_1) and the long track (I_3). Let the adversary play I_1 with probability q and I_3 with probability $1 - q$.

By the **Principle of Indifference**, we want to choose q such that the algorithm designer is indifferent between their best “buy early” strategy (A_1) and their best “buy late” strategy (A_4). We set their expected ratios equal:

$$\mathbb{E}[A_1] = \mathbb{E}[A_4] \implies q(2) + (1 - q)(1) = q(1) + (1 - q) \left(\frac{3}{2} \right)$$

Solving for q :

$$1 + q = 1.5 - 0.5q \implies 1.5q = 0.5 \implies q = \frac{1}{3}$$

Thus, the optimal worst-case random instance is:

$$I = \left\{ \begin{array}{ll} I_1 & \text{with probability } \frac{1}{3} \\ I_2 & \text{with probability } 0 \\ I_3 & \text{with probability } \frac{2}{3} \end{array} \right\}$$

The expected competitive ratio is a weighted average over the possible instances. The table below shows the expected ratios of all deterministic algorithms against this specific random instance I :

	A_1	A_2	A_3	A_4
$\mathbb{E}_I \left[\frac{A(I)}{\text{OPT}(I)} \right]$	$\frac{4}{3}$	$\frac{5}{3}$	$\frac{5}{3}$	$\frac{4}{3}$

Against this adversary distribution, the absolute best a deterministic algorithm can do is a ratio of $4/3$ (achieved by choosing either A_1 or A_4).

Because $\min_A \mathbb{E}_I \left[\frac{A(I)}{\text{OPT}(I)} \right] = \frac{4}{3}$, Yao's Principle dictates that the right-hand side of the inequality evaluates to $4/3$. Therefore, we have rigorously proven that no randomized algorithm can have an expected competitive ratio better than $4/3$ for this scenario.